



Ch 2: Knowledge Representation and Reasoning

Artificial Intelligence

Dr. Udaya R. Dhungana
Assistant Professor
Pokhara University, Nepal

Former Guest Faculty, Darmstadt University of Applied Sciences, Germany



Overview

Ch 2: Knowledge Representation and Reasoning (7 hrs)

1. First-Order Logic (FOL) and reasoning
2. Probabilistic reasoning
 - 2.1.1. Hidden Markov Models
 - 2.1.2. Dynamic Bayesian Networks
3. Ontological Engineering
4. Semantic Networks
5. Description Logic
6. Fuzzy Logic-
 - 6.1.1. Fuzzy Inference Systems
 - 6.1.1.1. Mamdani
 - 6.1.1.2. Sugeno
 - 6.1.1.3. Tsukamoto
 - 6.1.2. Defuzzification techniques

1. Time and Uncertainty
2. Transition Models and Sensor Models
3. Temporal Model and Inference
4. Types of Temporal Models
 - 4.1. Hidden Markov Models
 - 4.2. Dynamic Bayesian Networks

PROBABILISTIC REASONING OVER TIME

Time and Uncertainty

- **Probability in Static World**
 - each random variable has a single fixed value
 - Example: **Repairing a car**
 - **Evidence:** whatever is broken remains broken during the process of diagnosis
 - **Aim:** is to infer the state of the car from observed evidence, which also remains fixed.
- **Probability in DYNAMIC World**
 - each random variable have different values over time
 - Example: **Treating a diabetic patient**
 - **Evidences:** such as recent insulin doses, food intake, blood sugar measurements and so on
 - **Aim:** using the evidence, make a decision about the patient's food intake and insulin dose
 - **HOWEVER** the Blood sugar levels can change rapidly over time, depending on recent food intake and insulin doses, metabolic activity, the time of day, and so on
- How can dynamic situations like these be modeled?



Time and Uncertainty

- **Time**

- In probabilistic reasoning over time, the state of the world is considered to evolve over discrete time steps $t = 0, 1, 2, \dots$
- At each time step, a system can be in a particular state.

- **Uncertainty:**

- Real-world systems are uncertain due to:
 - *Partial observability* (we may not fully observe the current state).
 - *Stochastic dynamics* (the next state depends probabilistically on the current state).
 - *Noisy sensors* (observations may not perfectly reflect the true state).
- This leads to the need for **temporal probabilistic models** like Markov models and Dynamic Bayesian Networks (DBNs).



Time and Uncertainty

- Consider **IF**
 - X_t = set of state variables at time t (assumed to be unobservable)
 - E_t = set of observable evidence variables
 - e_t = is the **actual observed value** (a specific value observed) of that evidence at time t , **THEN**
 - The observation at time t is $E_t = e_t$ for some set of values e_t
- Example:
 - In a weather monitoring system, suppose:
 - E_t = sensor reading at time t
 - e_t = “rain” (the observed value at that time)
 - $\therefore E_t = e_t$ means “at time t , the observed evidence is rain.”



Example: Umbrella and Rain

- **Look at a scenario:** You're a security guard trying to figure out **whether it's raining** outside. But you can't go out, so you rely on **observations**—like seeing if your director arrives with an umbrella.
- **Representing the Problem**
 - **State variable** $X_t \rightarrow$ what's really happening
 - For example, R_t : is it raining? [R_t = actual rain status at time t]
 - **Evidence variable** $E_t \rightarrow$ what you observe
 - For Example, U_t : is the director carrying an umbrella? [U_t = observed umbrella presence at time t]
 - **Start Times**
 - State starts at $t = 0$
 - Observations (evidence) start at $t = 1$

Notation:

$a : b$ = sequence from a to b , including both

$X_{a:b}$ = variables from X_a to X_b

Example: $U_{1:3} = U_1, U_2, U_3$



Transition Model and Sensor Model

- Given the set of state and evidence variables for a given problem
 - the next step is to specify how the world evolves (the **Transition model**)
 - This model describes how the state of a system changes over time
 - how the evidence variables get their values (the **Sensor model**).
 - It describes how observations are generated based on the system's state
- These models are crucial for understanding and predicting the behavior of dynamic systems in various applications, including
 - tracking the location of a robot
 - tracking the economic activity of a nation
 - making sense of a spoken or written sequence of words



Transition Model

- It captures how the system evolves over time.
- The transition model specifies the *probability distribution over the latest state variables, given the previous values*, i.e.

$$P(X_t | X_{0:t-1})$$

- **Problem:** $X_{0:t-1}$ is unbounded in size as t increases.

X_t = the state at time t and
 X_{t-1} = the state at the previous time step

- **Solution:**
 - Using **Markov assumption**- “the current state depends on only a *finite fixed number* of previous states.” (Also called **Markov processes** or **Markov chains**)
 - **First-order Markov process:** the current state depends only on the previous state and not on any earlier states (i.e. a state provides enough information to make the future conditionally independent of the past)
- **Therefore**, in a first-order Markov process, the transition model is the probability(conditional) distribution over the next state given the current state

$$\therefore P(X_t | X_{t-1})$$

- **Example:** In a robot navigation problem, the transition model might represent the probability of moving to a new location given the current location and an action

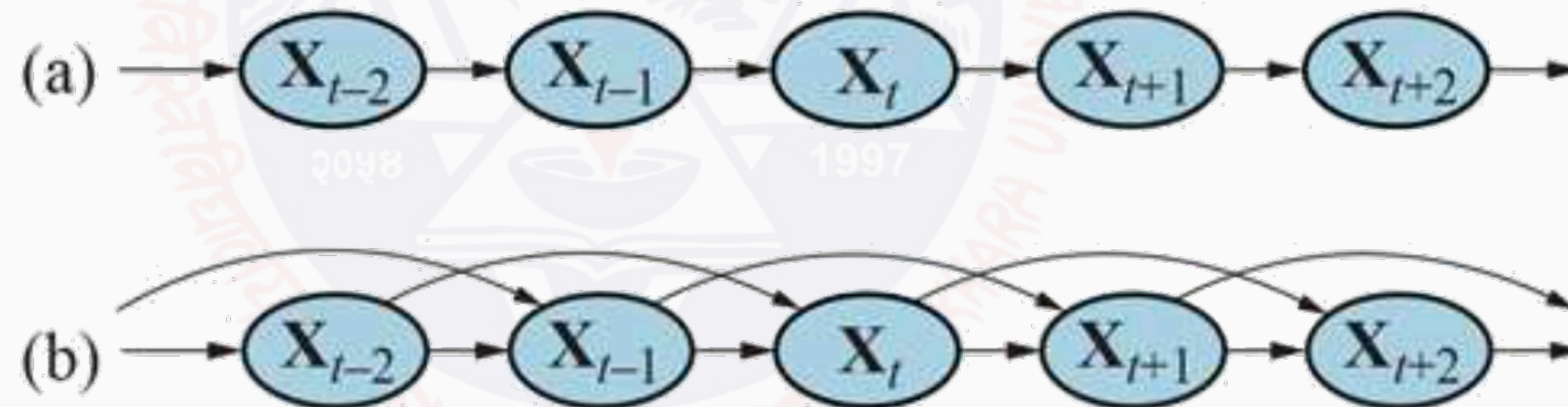


Transition Model

- The transition model for a **second-order Markov process** is the conditional distribution

$$P(X_t | X_{t-2}, X_{t-1})$$

•



(a) Bayesian network structure corresponding to a first-order Markov process with state defined by the variables X_t . (b) A second-order Markov process.



Transition Model

- Still **Problem** in Markov Assumption:
 - there are infinitely many possible values of t .
 - Do we need to specify a different distribution for each time step?
- **Solution:**
 - assume that changes in the world state are caused by a **time-homogeneous** process (i.e. a process of change that is governed by laws that do not themselves change over time)
- **Example:**
 - In the umbrella world, then, the conditional probability of rain,

$P(R_t | R_{t-1})$ is the same for all t .



Sensor Model

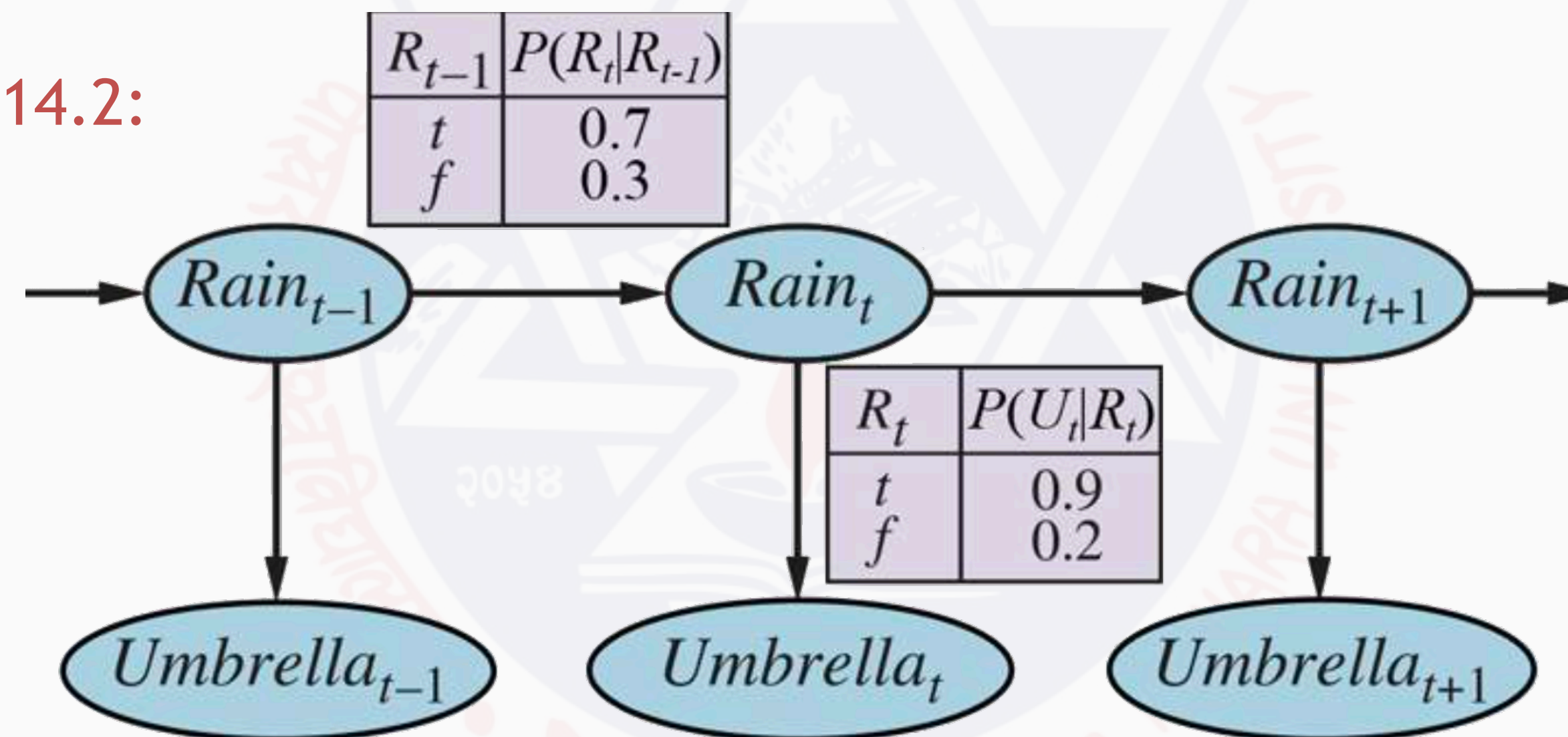
- A **sensor model** (also called *observation model*), assuming first order Markov process,
 - defines the **probability of an observation** given the current state:
$$P(E_t | X_t)$$
where E_t is the evidence (observation) at time t , and X_t is the hidden state at that time.
 - describes the likelihood of observing certain evidence given the hidden state
- **Example:**
 - A robot's camera may provide a noisy image (evidence) of the environment.
 - The sensor model estimates how likely a certain image is given the robot's location.



Bayesian Network View for Umbrella Rain World

- The transition and sensor models can be described using a **Bayesian network**.

Fig: 14.2:



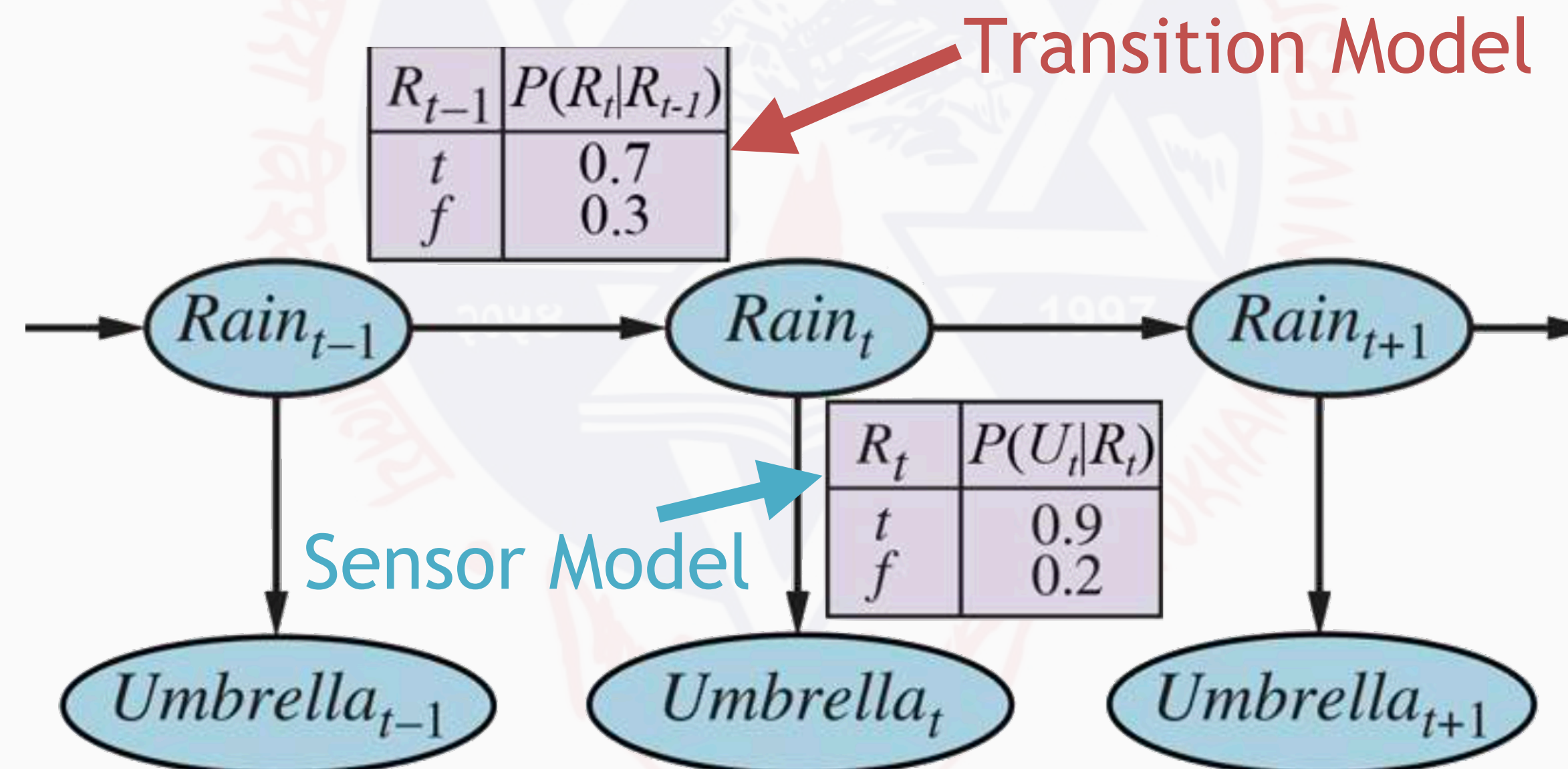
Bayesian network structure and conditional distributions describing the umbrella world. The transition model is $\mathbf{P}(Rain_t | Rain_{t-1})$ and the sensor model is $\mathbf{P}(Umbrella_t | Rain_t)$.



Bayesian Network View for Umbrella Rain World

- We have a specification of the complete joint distribution:

$$P(\mathbf{X}_{0:t}, \mathbf{E}_{1:t}) = P(\mathbf{X}_0) \prod_{i=1}^t P(\mathbf{X}_i | \mathbf{X}_{i-1}) P(\mathbf{E}_i | \mathbf{X}_i)$$



Complete Joint Distribution

- We have a specification of the complete joint distribution over all the variables, using

$$P(\mathbf{X}_{0:t}, \mathbf{E}_{1:t}) = P(\mathbf{X}_0) \prod_{i=1}^t P(\mathbf{X}_i | \mathbf{X}_{i-1}) P(\mathbf{E}_i | \mathbf{X}_i)$$

- the **joint probability** over a sequence of hidden states and observations in a **temporal (dynamic) model** like a **Hidden Markov Model (HMM)** or **Dynamic Bayesian Network (DBN)**
- Forms the foundation for filtering, prediction, smoothing in time-series models like HMMs.
- Used for **inference** when you're trying to figure out hidden states from observations.



Sensor Model

- We have a specification of the complete joint distribution over all the variables, using

$$P(\mathbf{X}_{0:t}, \mathbf{E}_{1:t}) = P(\mathbf{X}_0) \prod_{i=1}^t P(\mathbf{X}_i | \mathbf{X}_{i-1}) P(\mathbf{E}_i | \mathbf{X}_i)$$

Where $P(X_0)$ = the initial state model

$P(X_i | X_{i-1})$ = the transition model

$P(E_i | X_i)$ = the sensor model

- This equation defines the semantics of the family of temporal models represented by the three terms



Temporal Model

- A **Temporal Model** is a probabilistic model that represents how a system evolves over time.
- It captures the **dynamics** of hidden or observable variables across time steps.
- A temporal model **connects variables at different times** – for example, how today's weather influences tomorrow's weather.
- Examples of Temporal Model
 - Hidden Markov Model (HMM)
 - Dynamic Bayesian Network (DBN)
 - Kalman Filter
 - Markov Chains



Structure of Temporal Model

- A Temporal Model typically includes

- State Transition Model

- $P(X_t | X_{t-1})$: how the system evolves from one time step to the next.

- Sensor/Observation Model

- $P(E_t | X_t)$: how observations are generated from the current state

- Initial State Distribution

- $P(X_0)$: your belief about the system at time 0.

- Why Temporal Model?

- Predict future events: $P(X_{t+1} | History)$

- Infer hidden states: $P(X_t | E_{1:t})$

- Smooth past estimates: $P(X_k | E_{1:t})$ for $k < t$

- Temporal Model in Umbrella-Rain World

- Transition model: $P(Rain_t | Rain_{t-1})$

- Sensor Model: $P(Umbrella_t | Rain_t)$



Inference in Temporal Model

- Having set up the structure of a generic temporal model, we can formulate the basic inference tasks that must be solved:
 - Filtering or state estimation
 - Prediction
 - Smoothing
 - Most Likely Explanation



Filtering or State Estimation

- Compute the current state belief given all past evidence
- is the task of computing the **belief state** $P(X_t | e_{1:t})$ - the posterior distribution over the most recent state given all evidence to date
- In the umbrella example, this would mean computing the probability of rain today, given all the umbrella observations made so far



Prediction

- is the task of computing the posterior distribution over the *future* state, given all evidence to date.
 - That is, we wish to compute $P(X_{t+k} | e_{1:t})$ for some $k > 0$
- In the umbrella example,
 - this might mean computing the probability of rain three days from now, given all the observations to date



Smoothing

- This is the task of computing the posterior distribution over a *past* state, given all evidence up to the present
 - That is, we wish to compute $P(X_k | e_{1:t})$ for some $0 \leq k < t$
- In the umbrella example, it might mean computing the probability that it rained last Wednesday, given all the observations of the umbrella carrier made up to today



Most Likely Explanation

- Given a sequence of observations/evidences,
 - find the sequence of states that is most likely to have generated those observations: $\text{argmax}_{x_{1:t}} P(x_{1:t} | e_{1:t})$
- Example:
 - if the umbrella appears on each of the first three days and is absent on the fourth, then the most likely explanation is that it rained on the first three days and did not rain on the fourth.
- Some Applications:
 - In speech recognition—where the aim is to find the most likely sequence of words, given a series of sounds
 - In the reconstruction of bit strings transmitted over a noisy channel



Summary- Inference in Temporal Models

Inference Task	What it computes	Equation	Typical Use
Filtering	Current State	$P(X_t e_{1:t})$	Real-time tracking (e.g., robot localization)
Prediction	Future State (s)	$P(X_{t+k} e_{1:t}) \quad k > 0$	Forecasting (e.g., weather prediction)
Smoothing	Past State(s)	$P(X_k e_{1:t}) \quad 0 \leq k < t$	Retrospective diagnosis
Most likely Explanation	Most likely state sequence	$\operatorname{argmax}_{x_{1:t}} P(x_{1:t} e_{1:t})$	Speech recognition, tagging



Inference in Temporal Models

- Assignment:
 - Go through the subsection 14.2.1 (Text Book: Artificial Intelligence- A Modern Approach) and summarize the inference process of Filtering and Prediction for the Umbrella-Rain World example.



Hidden Markov Model (HMM)

- An HMM is a temporal probabilistic model in which the state of the process is described by a *single, discrete* random variable.
- Assume that the state of process is described by a single discrete random variable X_t (there is also a single evidence variable E_t). This is called a hidden Markov model (HMM)
- The umbrella example described is therefore an HMM, since it has just one state variable: $Rain_t$
- What happens if you have a model with two or more state variables?
 - Combine the variables into a single “megavariable” whose values are all possible tuples of values of the individual state variables.
- Although HMMs require the *state* to be a single, discrete variable, there is no corresponding restriction on the *evidence* variables.



Hidden Markov Model (HMM)

- This restricted model allows for a simple and elegant **matrix implementation** of all the basic algorithms
- Let the state variable X_t have values $1, \dots, S$ where S is the number of possible states,
- The transition model
 - The transition model $P(X_t | X_{t-1})$ becomes an $S \times S$ matrix T , where
 - $T_{ij} = P(X_t = j | X_{t-1} = i)$
 - T_{ij} is the probability of a transition from state i to state j .
- The Sensor Model
 - We also put the sensor model $P(E_t | X_t)$ in matrix form
 - We know the value of the evidence variable E_t is known at time t (call it e_t), we need only specify, for each state, how likely it is that the state causes e_t to appear: we need $P(E_t = e_t | X_t = i)$ for each state i . We place these values into an $S \times S$ diagonal matrix O_t , one for each step.
 - $O_{t(ij)} = P(E_t = e_t | X_t = i)$



Hidden Markov Model (HMM)

- Types of Inference in HMM

Task	Goal	Example
Filtering	$P(X_t O_{1:t})$	Estimate current state
Prediction	$P(X_{t+1} O_{1:t})$	Predict future state
Smoothing	$P(X_k O_{1:T}), k < T$	Infer past states
Most likely explanation	$\arg \max_{X_{1:T}} P(X_{1:T} O_{1:T})$	Find most likely state sequence (Viterbi)



HMM- Umbrella- Rain Example

- Hidden State
 - $Rain_t \in \{true, false\}$ (Whether it's raining on day t)
- Observation
 - $Umbrella_t \in \{true, false\}$ (Whether someone carries an umbrella on day t)
- Model Parameters
 - Transition Model

$$P(Rain_t | Rain_{t-1}) = \begin{bmatrix} P(Rain_t = true | Rain_{t-1} = true) & = 0.7 \\ P(Rain_t = true | Rain_{t-1} = false) & = 0.3 \end{bmatrix}$$

- Sensor Model

$$P(Umbrella_t = true | Rain_t = true) = 0.9$$
$$P(Umbrella_t = true | Rain_t = false) = 0.2$$

- Prior

$$P(Rain_0 = true) = 0.5$$



HMM- Umbrella- Rain Example

- **Filtering:** What's the probability it's raining today, given umbrella observations up to today?

- Lets say observation for 2 days:

- Umbrella_1 = true
- Umbrella_2 = true

- Compute:

$$P(Rain_2 \mid Umbrella_1 = true, Umbrella_2 = true)$$

- How?: Use forward algorithm (Update belief day by day with the recursive formula):

$$P(Rain_t \mid e_{1:t}) = \alpha P(e_t \mid Rain_t) \sum_{Rain_{t-1}} P(Rain_t \mid Rain_{t-1}) P(Rain_{t-1} \mid e_{1:t-1})$$



HMM- Umbrella- Rain Example

- **Prediction:** What is the chance it will rain tomorrow, given umbrella evidence up to today?
- Lets say observation for 2 days:
 - Umbrella_1 = true
 - Umbrella_2 = true
- Compute:
$$P(Rain_3 \mid Umbrella_1 = true, Umbrella_2 = true)$$
- How?: Extend forward algorithm one more step without using Umbrella_3:

$$P(Rain_3) = \sum_{Rain_2} P(Rain_3 \mid Rain_2) P(Rain_2 \mid e_{1:2})$$



Prediction Implementation- Umbrella- Rain Example

- Hidden State: $Rain_t \in \{true, false\}$
- Observations: $Umbrella_t \in \{true, false\}$
- Transition Model $P(Rain_t | Rain_{t-1})$:
 - $P(Rain_t = true | Rain_{t-1} = true) = 0.7$
 - $P(Rain_t = true | Rain_{t-1} = false) = 0.3$
- Observation Model $P(Umbrella_t | Rain_t)$:
 - $P(Umbrella_t = true | Rain_t = true) = 0.9$
 - $P(Umbrella_t = true | Rain_t = false) = 0.2$
- Initial Belief
 - $P(Rain_0 = true) = 0.5$



Prediction Implementation- Umbrella- Rain Example

- Forward algorithm up to time $t = 2$: compute $P(Rain_2 | e_{1:2})$
- Prediction step: compute $P(Rain_3 | e_{1:2})$ using the transition model

$$P(Rain_3) = \sum_{Rain_2} P(Rain_3 | Rain_2) \cdot P(Rain_2 | e_{1:2})$$



Prediction Implementation- Umbrella- Rain Example

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  // Index: 0 = false, 1 = true
6
7  // Transition probabilities
8  const double transition[2][2] = {
9      {0.7, 0.3}, // From Rain_t=true
10     {0.3, 0.7}  // From Rain_t=false
11 };
12
13 // Observation probabilities
14 const double observation[2][2] = {
15     {0.9, 0.1}, // P(Umbrella=true | Rain=true/false)
16     {0.2, 0.8}
17 };
18
19 // Normalize a 2-element probability vector
20 void normalize(vector<double>& prob) {
21     double total = prob[0] + prob[1];
22     prob[0] /= total;
23     prob[1] /= total;
24 }
```

```
26 int main() {
27     // Prior belief at t=0
28     vector<double> belief = {0.5, 0.5}; // [Rain=false, Rain=true]
29
30     // Observations: Umbrella=true on day 1 and 2
31     vector<bool> observations = {true, true};
32
33     // FORWARD: filtering step
34     for (bool umbrella : observations) {
35         vector<double> prediction(2, 0.0);
36         // Prediction step: belief_t =  $\sum P(X_t | X_{t-1}) * belief_{t-1}$ 
37         for (int curr = 0; curr < 2; ++curr) {
38             for (int prev = 0; prev < 2; ++prev) {
39                 prediction[curr] += transition[prev][curr] * belief[prev];
40             }
41         }
42
43         // Correction step: multiply by sensor model P(E_t | X_t)
44         for (int i = 0; i < 2; ++i) {
45             prediction[i] *= observation[i][umbrella ? 0 : 1];
46         }
47
48         normalize(prediction);
49         belief = prediction;
50     }
```



Prediction Implementation- Umbrella- Rain Example

```
52 cout << "Belief after day 2 (filtering):" << endl;
53 cout << "P(Rain=false) = " << belief[0] << endl;
54 cout << "P(Rain=true) = " << belief[1] << endl;
55
56 // PREDICTION: for day 3
57 vector<double> prediction(2, 0.0);
58 for (int curr = 0; curr < 2; ++curr) {
59     for (int prev = 0; prev < 2; ++prev) {
60         prediction[curr] += transition[prev][curr] * belief[prev];
61     }
62 }
63
64 cout << "\nPrediction for day 3:" << endl;
65 cout << "P(Rain=false) = " << prediction[0] << endl;
66 cout << "P(Rain=true) = " << prediction[1] << endl;
67
68 return 0;
69 }
```

Output:

```
Belief after day 2 (filtering):
P(Rain=false) = 0.883357
P(Rain=true) = 0.116643
```

```
Prediction for day 3:
P(Rain=false) = 0.653343
P(Rain=true) = 0.346657
```



Dynamic Bayesian Networks (DBN)

- DBN is a type of **Bayesian Network** that models **stochastic processes over time**
- It is an extension of a Bayesian Network (BN)
- DBNs are powerful tools for representing *stochastic processes* where the system evolves over time, such as
 - speech recognition
 - activity monitoring
 - robot localization
 - weather prediction etc.
- A DBN is essentially a **Bayesian network extended over time**, where each "slice" represents the system's state at one time step



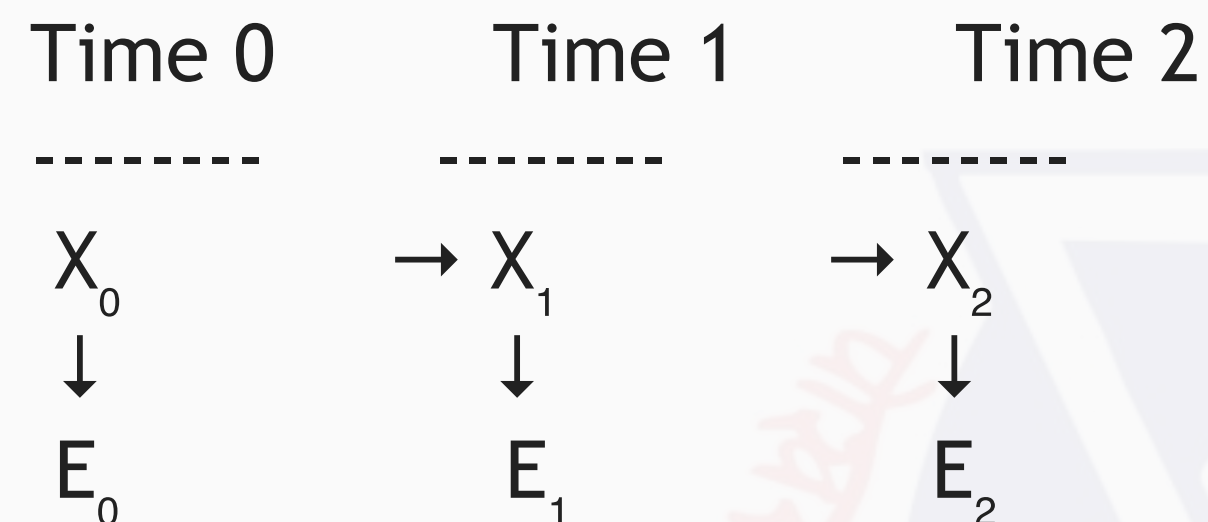
2-Time-Slice Bayesian Network (2-TBN)

- 2-Time-Slice Bayesian Network (2-TBN)
 - is a Bayesian network fragment that defines the probabilistic dependencies between variables across two consecutive time slices, typically denoted as time $t-1$ and t .
 - is the core building block of a Dynamic Bayesian Network (DBN)
- Let us consider $X_t = \{X_t^1, X_t^2, \dots\}$ are the state variables and $E_t = \{E_t^1, E_t^2, \dots\}$ are observation variables, then 2-TBN consists of
 - A prior distribution over initial state variables: $P(X_0)$
 - A transition model: $P(X_t | X_{t-1})$ and
 - An observation model: $P(E_t | X_t)$



Structure of a 2-TBN

- The 2-TBN is replicated at every time step to form the full DBN:



- Nodes for variables in **time slice** $t - 1$ and t
- Directed edges:
 - From X_{t-1} to X_t : temporal dependencies
 - Within X_t : intra-slice dependencies
 - From X_t to E_t : observation/sensor model



Full DBN = Prior + Repeated 2-TBN

Component	Description
Prior	Bayesian network at time $t = 0$ (e.g., $P(X_0)$)
2_TBN	Defines how X_t and E_t depend on X_{t-1}
Unrolling	The 2-TBN is cloned for all $t = 1$ to T



Dynamic Bayesian Networks (DBN)

- A DBN is a probabilistic graphical model that represents a sequence of variables over time
- In a DBN, each time slice contains a set of variables and their probabilistic dependencies.
- It is essentially a Bayesian network unfolded across time.
- A DBN allows:
 - Inference over time (e.g., filtering, prediction, smoothing)
 - Learning temporal models from data
 - Modeling hidden and observed states



Components of a DBN

- **State Variables (X_t)**
 - Variables representing the system's state at time t
- **Observation Variables (E_t)**
 - Observed values at time t , depending on the hidden state.
- **Initial Model- $P(X_0)$**
 - The prior distribution over the state at time 0
- **Transition Model $P(X_t | X_{t-1})$**
 - How the state evolves from one time to the next.
- **Sensor Model $P(E_t | X_t)$**
 - Probability of observing evidence E_t given the current state.



Umbrella DBN (Rain Detection using Umbrella Observations)

- This is a classic **Hidden Markov Model (HMM)** represented as a **DBN**, where
 - $Rain_0, Rain_1$: Hidden state
 - (whether it is raining on day 0 and day 1)
 - $Umbrella_1$: Observation
 - (whether an umbrella is seen on day 1)
 - Prior Probability:
 - $P(R_0 = true) = 0.7$
 - $P(R_0 = false) = 0.3$

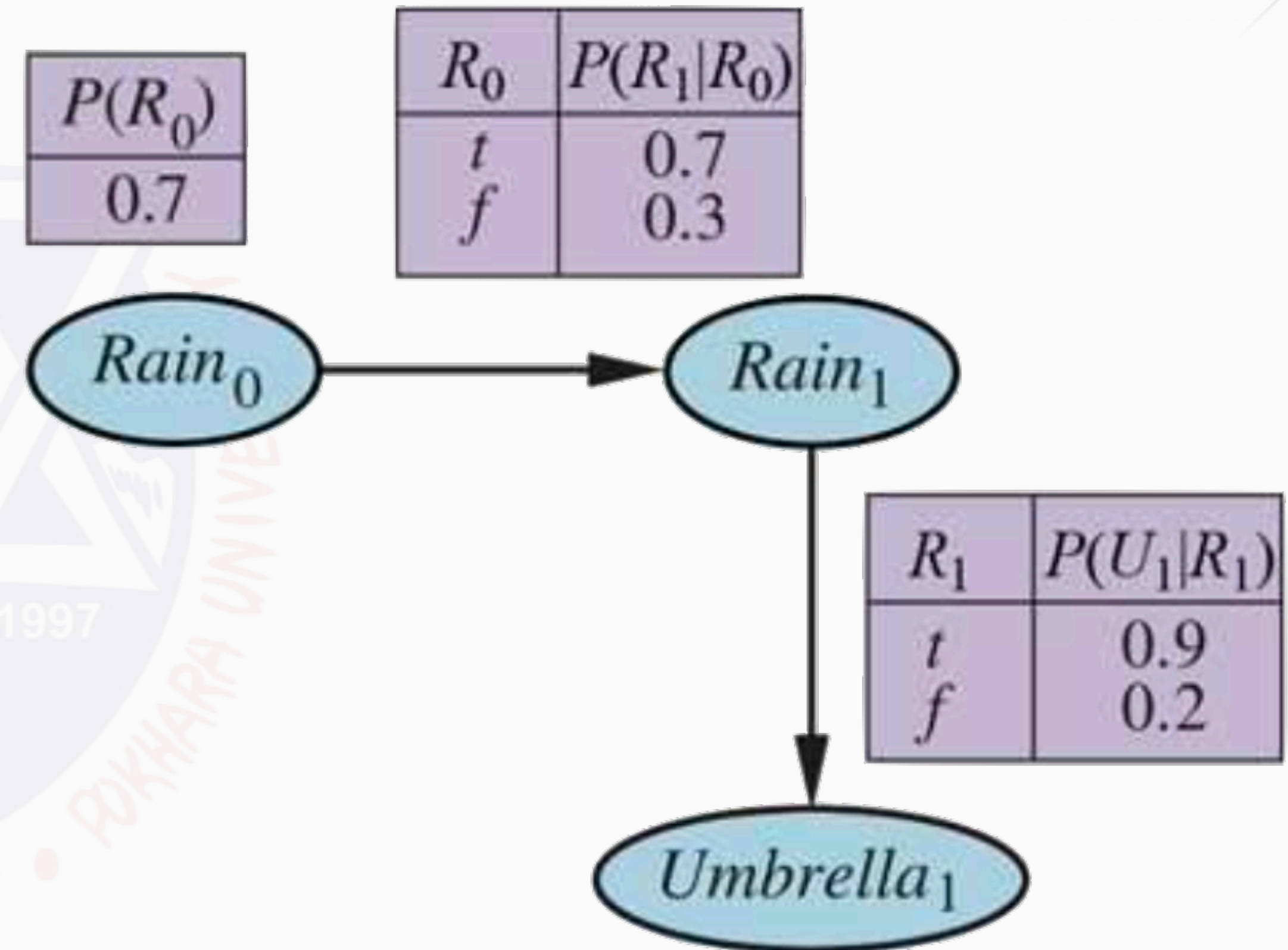


Fig. Specification of the prior, transition model, and sensor model for the umbrella



Umbrella DBN (Rain Detection using Umbrella Observations)

- **Prior Probability:**

$$P(R_0 = \text{true}) = 0.7$$

$$P(R_0 = \text{false}) = 0.3$$

- **Transition Model:**

$$P(R_1 = \text{true} \mid R_0 = \text{true}) = 0.7$$

$$P(R_1 = \text{true} \mid R_0 = \text{false}) = 0.3$$

- **Sensor (Observation) Model:**

$$P(U_1 = \text{true} \mid R_1 = \text{true}) = 0.9$$

$$P(U_1 = \text{true} \mid R_1 = \text{false}) = 0.2$$

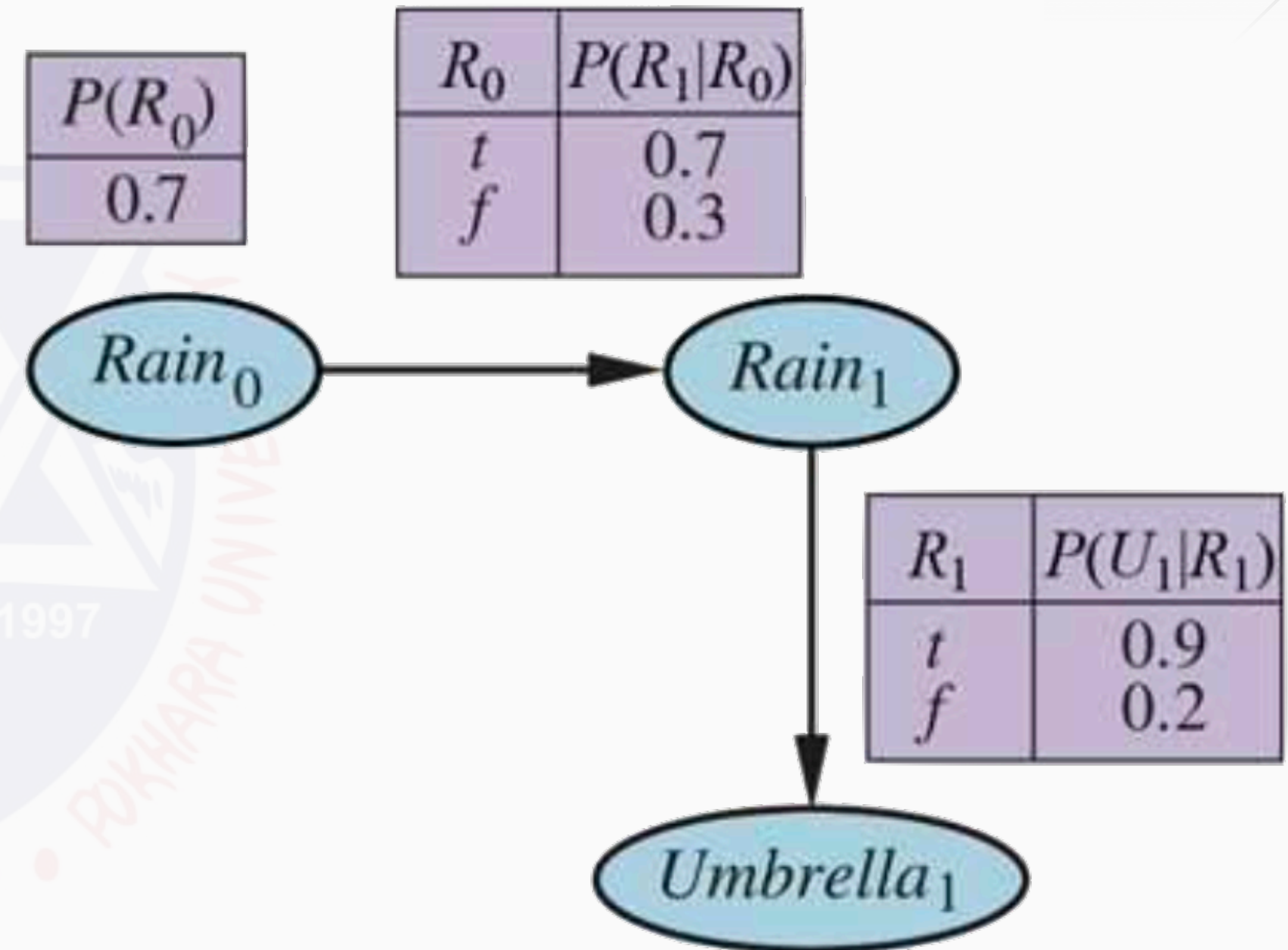


Fig. Specification of the prior, transition model, and sensor model for the umbrella



Umbrella DBN (Rain Detection using Umbrella Observations)

- **Rain** is the hidden state, and you cannot observe it directly.
- **Umbrella** is the observed variable, which depends on whether it's raining.
- This **DBN** models how the probability of rain evolves over time and how **observations** (umbrellas) give evidence about the hidden state.
- This is a **2-time slice DBN (2-TBN)**.
- Future time slices are assumed to repeat this structure.

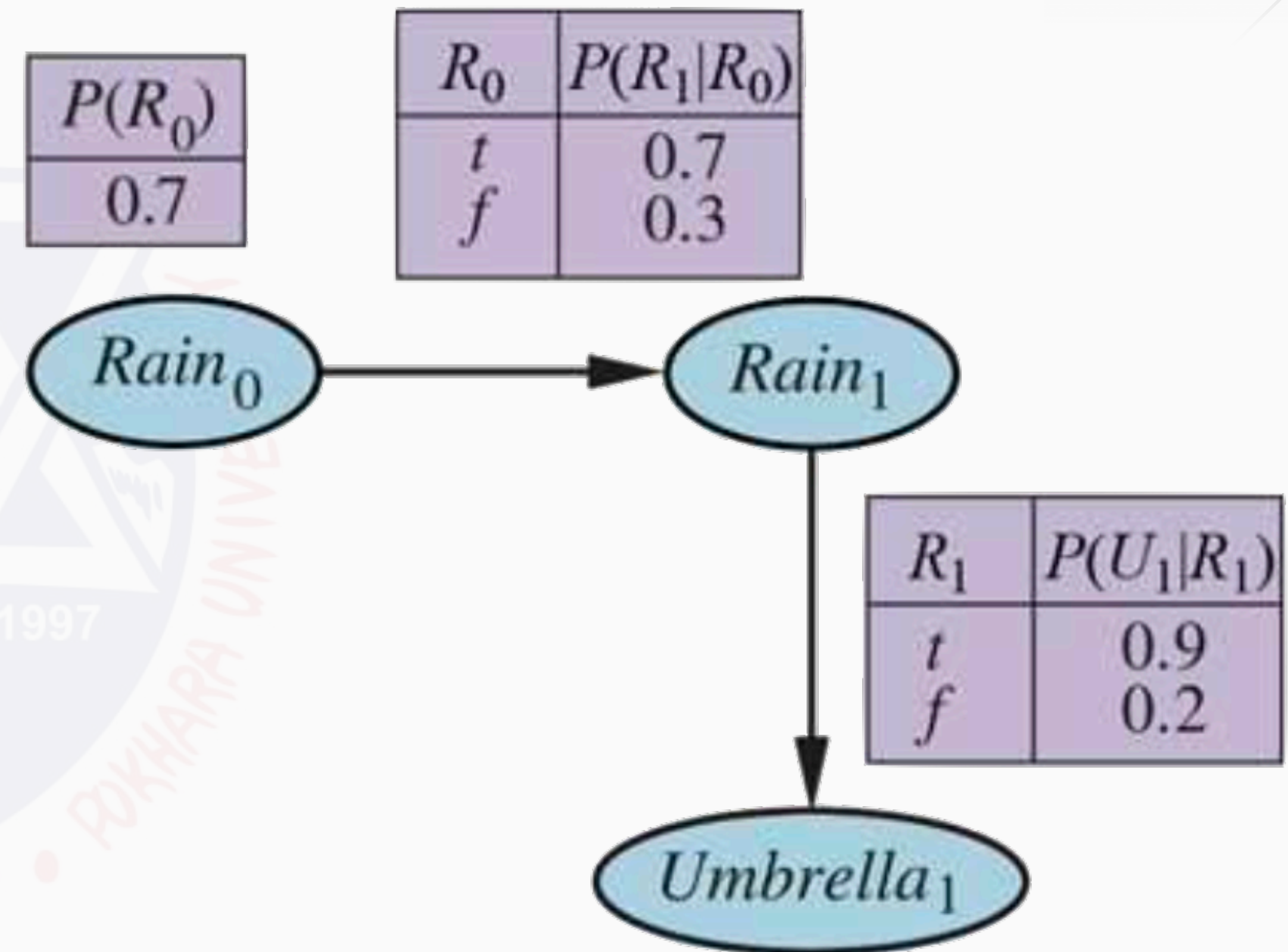


Fig. Specification of the prior, transition model, and sensor model for the umbrella



Implementation of Umbrella DBN

- This implementation does:
 - Prior initialization
 - Transition update
 - Observation likelihood
 - Filtering using Forward Algorithm
- C++ implementation of the Umbrella HMM as a DBN.

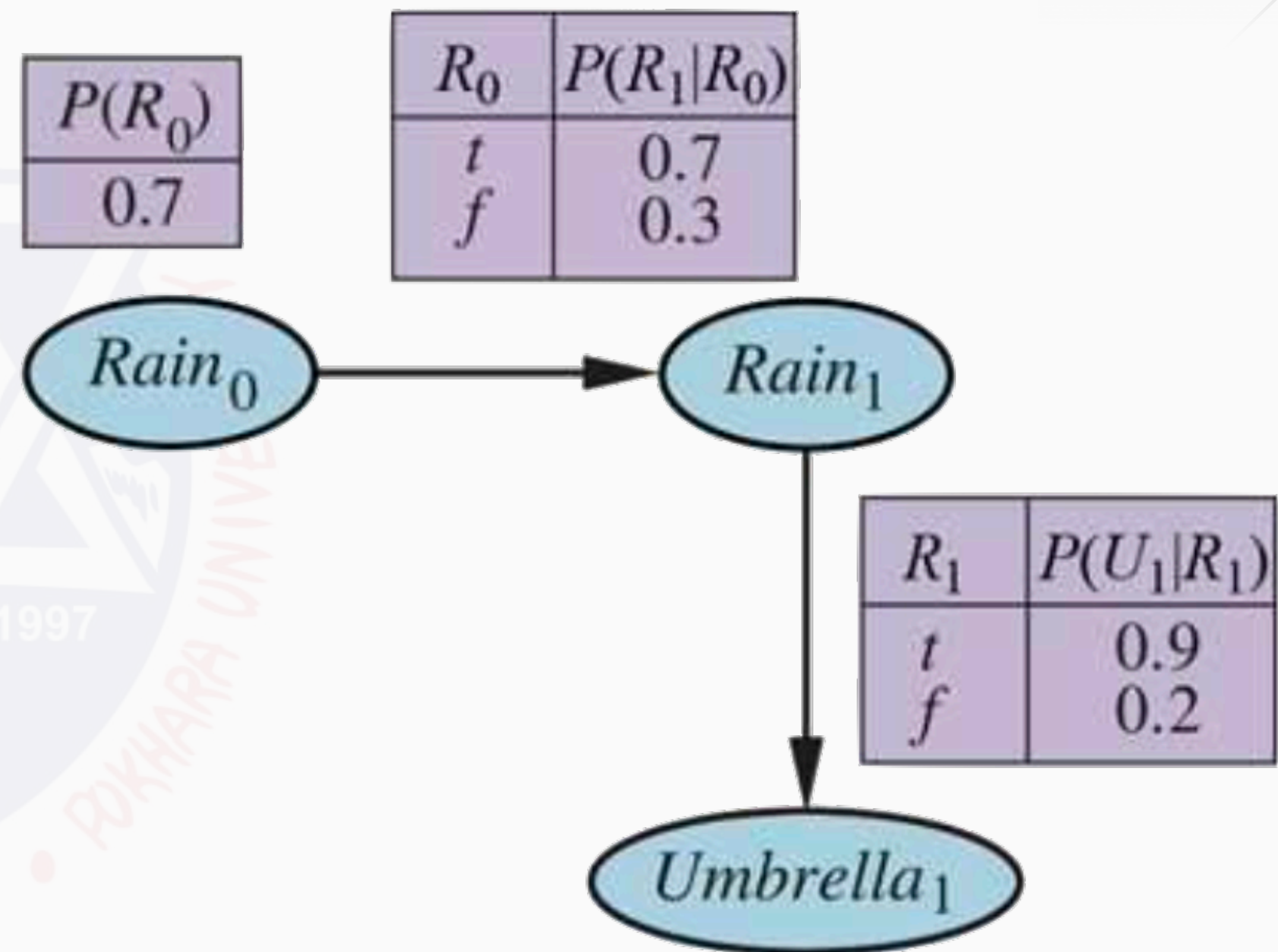


Fig. Specification of the prior, transition model, and sensor model for the umbrella



Implementation of Umbrella DBN

```
5  #include <iostream>
6  #include <vector>
7  using namespace std;
8
9  // Rain state: true = raining, false = not raining
10 enum RainState { FALSE, TRUE };
11
12 // Forward message for filtering
13 struct Belief {
14     double prob_true;
15     double prob_false;
16 };
17
18 class UmbrellaHMM {
19 public:
20     // Initial prior P(Rain_0)
21     Belief prior = {0.7, 0.3};
22
23     // Transition model P(Rain_t | Rain_t-1)
24     double transition[2][2] = {
25         {0.7, 0.3}, // From TRUE: P(TRUE), P(FALSE)
26         {0.3, 0.7}  // From FALSE
27     };
28
29     // Observation model P(U_t | Rain_t)
30     double sensor[2] = {0.9, 0.2}; // sensor[Rain_t]
31
32     // Forward filtering
33     Belief forward(Belief prev_belief, bool umbrella_observed) {
34         Belief predicted;
35         predicted.prob_true =
36             transition[TRUE][TRUE] * prev_belief.prob_true +
37             transition[FALSE][TRUE] * prev_belief.prob_false;
38
39         predicted.prob_false =
40             transition[TRUE][FALSE] * prev_belief.prob_true +
41             transition[FALSE][FALSE] * prev_belief.prob_false;
42
43         // Update with sensor evidence
44         double likelihood_true = umbrella_observed ? sensor[TRUE] : (1.0 - sensor[TRUE]);
45         double likelihood_false = umbrella_observed ? sensor[FALSE] : (1.0 - sensor[FALSE]);
46
47         Belief updated;
48         updated.prob_true = predicted.prob_true * likelihood_true;
49         updated.prob_false = predicted.prob_false * likelihood_false;
50
51         // Normalize
52         double norm = updated.prob_true + updated.prob_false;
53         updated.prob_true /= norm;
54         updated.prob_false /= norm;
55
56         return updated;
57     }
58 };
```



Implementation of Umbrella DBN

```
60  int main() {
61      UmbrellaHMM hmm;
62
63      // Umbrella observations over time
64      vector<bool> observations = {true, true, false, true};
65      Belief current = hmm.prior;
66
67      for (int t = 0; t < observations.size(); ++t) {
68          current = hmm.forward(current, observations[t]);
69          cout << "Day " << t+1 << " -> P(Rain=true): " << current.prob_true
70              << ", P(Rain=false): " << current.prob_false << endl;
71      }
72
73      return 0;
74  }
```

Output:

```
Day 1 -> P(Rain=true): 0.234818, P(Rain=false): 0.765182
Day 2 -> P(Rain=true): 0.126208, P(Rain=false): 0.873792
Day 3 -> P(Rain=true): 0.811919, P(Rain=false): 0.188081
Day 4 -> P(Rain=true): 0.270075, P(Rain=false): 0.729925
```



HMM is a special case of DBN

- HMM is a special case of DBN
- *When a DBN has:*
 - One hidden state variable
 - One observation variable
 - First-order Markov assumption
 - *then it reduces to an HMM*
- Assignment: Compare HMM vs DBN



Implementation of Umbrella DBN with *Prediction*

- **Prediction** means computing the probability of the hidden state at some future time step *without any observation*, using the **transition model** only.

Class Discussion and Demo





THANK YOU